
TECHNICAL WHITEPAPER

On-Chain Threat Intelligence Oracle

A Decentralised Framework for Bridging Web2 Attack Surface Monitoring with Web3 Infrastructure Security

Author:	James Kabingu Mbugua -- Interlock Resources
Version:	0.1 -- Initial Draft
Date:	May 2026
Website:	interlockr.github.io
GitHub:	github.com/interlockR
Licence:	MIT -- Open Source

Abstract

Approximately 80% of funds stolen from Web3 projects originate from attacks on Web2 infrastructure rather than smart contract exploits. Despite this, no decentralised threat intelligence infrastructure exists that Web3 protocols can natively query at runtime. This paper presents the On-Chain Threat Intelligence Oracle (OCTIO) -- a system architecture and implementation specification for a decentralised network that monitors Web2 attack indicators, stores verified intelligence immutably on a public blockchain, and exposes a native query interface to DeFi protocols via a Chainlink-compatible oracle adapter.

OCTIO introduces a token-based reputation and incentive mechanism governing submission quality, a multi-party validation scheme resistant to Sybil attacks, and a gas-efficient on-chain data structure optimised for high-frequency protocol queries. The system is designed to be deployed on Ethereum mainnet and L2 networks, with a monitoring layer compatible with Certificate Transparency feeds, DNS change APIs, phishing URL databases, and supply chain vulnerability feeds.

1. The Problem

1.1 The Web2 Attack Surface of Web3 Systems

The decentralised finance ecosystem holds tens of billions of dollars in total value locked across Ethereum, Layer 2 networks, and alternative chains. The security community has invested significant effort in smart contract auditing -- the formal and manual review of Solidity

code for on-chain vulnerability classes including reentrancy, access control failures, oracle manipulation, flash loan exploits, and arithmetic errors.

This investment has not proportionally reduced losses. Industry data consistently shows that the dominant attack vector is not the smart contract layer but the Web2 infrastructure surrounding it. The following incident categories account for the majority of documented DeFi losses by value:

DNS Hijacking: Attacker gains control of DNS records, redirecting traffic to malicious frontends. Example: Curve Finance (2022, 570,000 USD).

Frontend Supply Chain Compromise: Malicious code injected into npm packages used by DeFi frontends. Example: Ledger Connect Kit (2023, 600,000 USD).

Phishing and Social Engineering: Developer credentials compromised via phishing. Example: Ronin Network (2022, 625,000,000 USD).

BGP Hijacking: Internet routing manipulation redirecting traffic at the network layer. Example: MyEtherWallet (2018, 17,000,000 USD).

Cloud Misconfiguration: Exposed credentials enabling attacker infrastructure access. Example: Mixin Network (2023, 200,000,000 USD).

1.2 The Intelligence Gap

In every documented case above, observable signals preceded the loss event. Certificate Transparency logs recorded the issuance of fraudulent certificates. DNS change feeds showed anomalous record modifications. Package registries showed unusual publish activity. Phishing URLs were submitted to public databases before the attacks peaked.

The problem is not the absence of signals. It is the absence of infrastructure to collect, verify, and distribute those signals to the systems that need them most -- the DeFi protocols and Web3 frontends that are the ultimate targets. Existing threat intelligence platforms are centralised, permissioned, and architecturally incompatible with decentralised systems. A smart contract cannot call VirusTotal. A DeFi protocol cannot query Shodan at transaction time.

1.3 The Requirement

What is required is threat intelligence infrastructure that is:

Decentralised and censorship-resistant -- no single operator can suppress or manipulate intelligence.

Queryable by smart contracts at runtime -- protocols can check threat context before executing.

Incentivised for quality and timeliness -- contributors are rewarded for accurate, early submissions.

Resistant to false or manipulated data -- bad submissions are economically penalised.

Economically sustainable -- the network operates without central funding.

No such system currently exists. OCTIO is designed to be that system.

2. System Overview

OCTIO operates as a four-layer system. Each layer is independently deployable and upgradeable.

Layer	Function	Technology
L1: Monitoring	Scans public Web2 sources for threat indicators	Python, Certstream, DNS APIs, PhishTank
L2: On-Chain Registry	Stores and governs verified threat intelligence	Solidity, Ethereum / Arbitrum
L3: Oracle Interface	Exposes threat data to querying protocols	Chainlink adapter, Solidity
L4: Dashboard	Public interface for intelligence visibility	React, Web3.js, Python API

3. Layer 1: Web2 Monitoring

3.1 Data Sources

Source	Data Type	Frequency
Certstream	Real-time SSL certificate issuance from CT logs	Real-time
PassiveDNS / SecurityTrails	DNS record changes for monitored domains	5 min poll
PhishTank / OpenPhish	Confirmed phishing URLs	1 hour poll
npm audit API	Package vulnerability and unusual publish events	15 min poll
Cloudflare Radar	BGP routing anomalies and traffic shifts	30 min poll

3.2 Indicator Schema

Each threat indicator is structured as follows before on-chain submission:

```
{ indicator_type: DNS_HIJACK | PHISHING_URL | CERT_ANOMALY | SUPPLY_CHAIN |  
  BGP_ANOMALY, target: keccak256(domain or address), severity: LOW | MEDIUM |  
  HIGH | CRITICAL, source: feed identifier string, evidence_hash: keccak256(raw  
  evidence data), timestamp: unix timestamp of detection, submitter: reporter  
  wallet address }
```

Domains and URLs are stored as keccak256 hashes rather than plaintext to prevent the registry from being used as a phishing link directory and to reduce on-chain storage costs by 60 to 90 percent.

4. Layer 2: On-Chain Registry

4.1 Contract Architecture

The on-chain layer consists of four smart contracts:

ThreatRegistry.sol -- Core indicator storage and public query interface.

ReputationManager.sol -- Researcher stake, reputation score, and reward distribution.

ValidationPool.sol -- Multi-party validation mechanism for submitted indicators.

GovernanceController.sol -- Protocol parameter governance via token-weighted voting.

4.2 ThreatRegistry Public Interface

```
function submitIndicator( bytes32 targetHash, uint8 indicatorType, uint8
severity, bytes32 evidenceHash ) external returns (uint256 indicatorId);
function isFlagged(bytes32 targetHash) external view returns (bool flagged,
uint8 maxSeverity); function getIndicator(uint256 indicatorId) external view
returns (Indicator memory); function getIndicatorsForTarget(bytes32 targetHash)
external view returns (uint256[] memory indicatorIds);
```

4.3 Validation Mechanism

Submitted indicators enter a 24-hour validation window. During this window, registered researchers vote to confirm or dispute the indicator. Voting weight is proportional to the voter's reputation score. An indicator reaches validated status when it accumulates three or more confirmations with combined reputation weight above the governance-set threshold.

Validated indicators are promoted to the active registry. Disputed indicators are reviewed by a randomly selected arbitration panel of high-reputation researchers. False submissions result in stake slashing for the original submitter. Correct disputes result in stake rewards for the disputing researcher.

4.4 Storage Optimisation

OCTIO uses the following optimisations to minimise gas costs:

Target hashes (keccak256, 32 bytes) instead of variable-length strings reduce storage by 60 to 90 percent for domain and URL indicators.

Packed struct layout fits indicator metadata into a single 256-bit storage slot where possible.

A Bloom filter provides $O(1)$ preliminary check before full indicator lookup, reducing negative query cost to approximately 2,100 gas.

Full evidence data is stored on IPFS with only the content hash stored on-chain.

5. Token Economic Model

5.1 The INTEL Token

The INTEL token is the native utility and governance token of the OCTIO network. It serves three functions: staking (researchers must stake INTEL to submit indicators), rewards (validated indicators earn INTEL from the protocol treasury), and governance (INTEL holders vote on protocol parameters).

5.2 Incentive Structure

Action	Token Effect
Submit valid indicator	+10 INTEL reward upon validation
Submit false indicator	-50 INTEL stake slash
Confirm valid indicator	+2 INTEL reward
Dispute false indicator successfully	+25 INTEL reward
Protocol query (consumer)	0.001 INTEL fee per query
Governance vote	No cost -- 1 token = 1 vote

5.3 Sybil Resistance

The primary Sybil resistance mechanism is economic: creating multiple identities requires staking INTEL for each. An attacker attempting to overwhelm the validation pool must stake sufficient INTEL across all identities, and successful disputes slash all of them. A secondary mechanism is reputation bootstrapping -- new researchers have low reputation weight and their votes carry minimal influence until they accumulate a history of correct submissions.

5.4 Token Distribution

Allocation	Percentage	Purpose
Protocol Treasury	40%	Researcher rewards and operational funding
Founding Team	20%	4-year vesting with 1-year cliff
Early Contributors	15%	Researchers and validators in beta period
Community Reserve	15%	Governance-controlled allocation
Protocol Liquidity	10%	DEX liquidity for token utility

6. Layer 3: Oracle Interface

6.1 Chainlink Adapter Design

The oracle interface is implemented as a Chainlink External Adapter, allowing any protocol that supports Chainlink oracle requests to query the OCTIO registry. A consuming protocol makes a Chainlink oracle request specifying the target hash and query type. The Chainlink node calls the adapter, which queries the ThreatRegistry contract and returns a structured response. The consuming protocol receives the result in its fulfillment callback function.

6.2 Consumer Integration

A DeFi protocol integrates OCTIO with the following pattern:

```
contract ProtocolWithOCTIO is ChainlinkClient { bytes32 private jobId; uint256
private fee; function checkCounterparty( address counterparty ) internal
returns (bytes32 requestId) { bytes32 targetHash = keccak256(
abi.encodePacked(counterparty) ); Chainlink.Request memory req =
buildChainlinkRequest( jobId, address(this), this.fulfill.selector );
req.add('targetHash', bytes32ToString(targetHash)); return
sendChainlinkRequest(req, fee); } function fulfill( bytes32 requestId, bool
flagged, uint8 severity ) public recordChainlinkFulfillment(requestId) { if
(flagged && severity >= CRITICAL) revert(); } }
```

6.3 Query Types

Query Type	Input	Output
Address check	Wallet or contract address	flagged (bool), severity (uint8)
Domain check	Domain string	flagged (bool), indicator count (uint256)
Certificate check	Certificate fingerprint hash	anomalous (bool), type (uint8)
Package check	npm package name and version	compromised (bool), CVE list

7. Layer 4: Dashboard

The public dashboard serves three audiences: security researchers monitoring the threat landscape, DeFi protocol teams assessing their exposure, and the general Web3 community tracking incident correlation.

7.1 Dashboard Components

Live Threat Feed: Real-time display of newly validated indicators with severity, type, and target hash.

Researcher Leaderboard: Reputation scores and submission history for top contributors.

Protocol Query Monitor: Live feed of oracle queries showing which protocols are checking what indicators.

Incident Correlation: Historical overlay mapping indicator spikes against documented Web3 hacks and financial losses.

Financial Markets Panel: Token price, treasury balance, and economic health metrics for the OCTIO network.

8. Security Analysis

8.1 Threat Model

OCTIO operates under the following threat model:

Adversarial Submitters: Researchers who submit false indicators to discredit targets or manipulate protocol behaviour. Mitigated by stake slashing and reputation penalties.

Sybil Attackers: Actors creating multiple identities to overwhelm the validation pool. Mitigated by stake requirements and reputation bootstrapping.

Governance Attacks: Token holders attempting to pass malicious parameter changes. Mitigated by timelocks on all governance actions and quorum requirements.

Oracle Manipulation: Attempt to manipulate Chainlink nodes returning false query results. Mitigated by multi-node aggregation and on-chain verification.

Smart Contract Exploits: Vulnerabilities in the registry or governance contracts. Mitigated by Foundry testing suite, Slither static analysis, and formal audit.

8.2 Limitations

OCTIO does not solve the oracle problem -- it extends it to a new domain. The system's security guarantees are only as strong as the honesty assumptions about the researcher network. A well-funded adversary controlling a majority of staked INTEL could theoretically manipulate the validation pool. This is a known limitation shared with all token-governance systems and is addressed through the stake slashing mechanism and the economic cost of majority control.

The monitoring layer provides best-effort coverage of public data sources. Sophisticated attackers who avoid leaving traces in monitored feeds will not be detected. OCTIO is a layer of defence -- it does not replace smart contract auditing, infrastructure hardening, or operational security practices.

9. Development Roadmap

Phase	Milestone	Target
Alpha	Monitoring layer live, contracts deployed on Sepolia testnet, dashboard operational	Q3 2026
Beta	INTEL token deployed, reputation system active, first 100 validated indicators	Q4 2026
v1.0	Mainnet deployment, Chainlink adapter live, first protocol integrations	Q1 2027
v1.5	L2 deployment (Arbitrum), expanded monitoring sources, governance activation	Q2 2027
v2.0	Cross-chain support, automated incident correlation, institutional partnerships	Q4 2027

10. Conclusion

The On-Chain Threat Intelligence Oracle addresses a fundamental gap in Web3 security infrastructure. Smart contracts are increasingly audited and hardened. The infrastructure surrounding them -- DNS, frontends, supply chains, cloud systems -- is not. OCTIO provides the missing primitive: a decentralised, incentivised, and natively queryable threat intelligence layer that Web3 protocols can use to check threat context before it costs them.

The system is not a complete solution to Web3 security. It is a new layer of defence that, combined with smart contract auditing, operational security, and infrastructure hardening, materially reduces the attack surface that accounts for most documented losses.

The most dangerous gaps in any system sit at intersections. OCTIO exists at the intersection of Web2 and Web3, of threat intelligence and decentralised finance, of cybersecurity and blockchain engineering. That is precisely where it needs to be.

References

- Breidenbach, L. et al. (2021). Chainlink 2.0: Next steps in the evolution of decentralised oracle networks. Chainlink Labs.
- Chainalysis. (2023). The Chainalysis 2023 Crypto Crime Report. Chainalysis Inc.
- Curve Finance. (2022). Post-mortem: DNS hijack incident, August 2022.
- Ledger. (2023). Ledger Connect Kit security incident report, December 2023.
- Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system.
- Sky Mavis. (2022). Ronin network security incident post-mortem.
- Wood, G. (2014). Ethereum: A secure decentralised generalised transaction ledger. Ethereum Project Yellow Paper.
- Zhou, L., Qin, K., and Gervais, A. (2023). SoK: Decentralised finance attacks. IEEE Symposium on Security and Privacy.

Disclaimer

This whitepaper is published for informational and academic purposes. It describes a proposed system architecture and does not constitute an offer to sell or a solicitation to buy any token or financial instrument. The INTEL token described herein has not been issued. All technical specifications are subject to change as the system design evolves. Nothing in this document should be construed as financial, legal, or investment advice.